Proof-Solving with COACH

Eylon Caplan

***Abstract*—Formal proof-solving is a difficult problem, and current methods struggle to automatically solve many types of proofs. The rise of large language models (LLM's) has enabled recent work that leverages the abundance of informal proof-solving data to help formal theorem-provers. Specifically, the Draft, Sketch, Prove framework by Jiang et al. [2022] uses LLM's to draft a proof, sketch it in a formal proof language, and have the prover attempt to prove it. In that method, repeating the process is often necessary to create a valid proof, but no feedback is given in subsequent attempts that could aid in future attempts. In this work, we use a simple problem to test various feedback mechanisms, including a module which we call COACH. From the results, we conclude that feedback is either unhelpful or outright harmful in this specific problem.**

I. Introduction

A. Formal Proof-Solving

The problem of formal proof-solving is specified by a knowledge base (KB) and a statement. In order to successfully prove a statement, a series of steps which each follow from the KB or previous steps must be enumerated and finally reach the statement. While a KB can be a 'bare minimum' of a mathematical system's axioms, it may also include already-proven theorems, and including these theorems often makes new proofs much faster Jiang et al. [2022].

Formal theorem provers exist, but have very limited capabilities. The search space of existing theorems is very large, but also the number of theorems that must be chained together is not known a priori, making search methods intractable for 'large gaps', or theorems that require many steps.

B. Interactive Theorem Provers

To remedy this, researchers have developed interactive theorem provers (ITP's), which are theorem provers that are guided by a human. In using ITP's, humans can outline the proof or limit the search, while still getting the result of a formal proof for a potentially long and complicated problem.

While ITP's are very useful, they still require large amounts of human time (working with a highly specialized mathematician) to work. Solving this issue was the goal of the Draft, Sketch, Prove (DSP) paper by Jiang et al. [2022].

C. Draft, Sketch, Prove

Formal proof-solving data is very scarce. This is because creating such data is far more intensive than normally solving a proof, along with the fact that there are many formal proof languages, and each disparate language comprises its own small dataset.

Unlike formal proof data, informal data is extremely abundant. An informal proof is a series of steps written in natural language that maps closely to the steps that could be taken in a formal proof. Much of this data exists in the form of textbooks and online proofs. With the recent success of LLM's, it has become reasonable to try to automate processes that were formerly only possible with humans, particularly when large data is present. The DSP method leverages LLM's success in informal proofs by first writing a proof 'draft', which is a proof in natural language. It then translates the draft into a formal proof language (Isabelle), leaving small gaps (the 'sketch'). Finally, it utilizes an off-the-shelf prover to close the gaps and verify the validity of the proof ('prove').

In this method, once a proof is verified as correct, it is guaranteed to be fully correct. Because of this, the authors were able to run the DSP process many times until a correct proof was reached. This was done either by increasing the number of drafts per problem, or increasing the number of sketches per draft. In both settings, the success of many repetitions relied on the variation introduced by the LLM's, and did not utilize information from previous proof attempts to inform new attempts.

The original DSP paper utilizes the miniF2F dataset, a collection of competition-style math questions ranging from middle to high-school level. Each problem consists of an informal problem (natural language), a formal problem (written in Isabelle), and a gold draft written by humans (which we do not use).

II. Method

The large number of drafts required to achieve some proofs seems to suggest that a better-quality draft, perhaps different than previous drafts, could be more successful than producing a large number of new drafts. In the DSP setting, the LLM is few-shot prompted with the natural language problem only, and has no access to information from previous failed proof attempts. In this project we close the loop by placing feedback from previous proof attempts into the initial problem prompt. We originally hypothesized that by providing feedback from failed proofs, the augmented DSP process would (1) solve the same

problems with fewer drafts and (2) solve problems previously unsolved by DSP.

In any such augmentation, the baseline (control) was always the standard DSP procedure with no feedback. However, we realized that it would be very difficult to compare to a baseline which has a 0% or 100% success rate. Because of this, we experimented with several problems until we found one that (1) has a success rate of about 27%, and (2) by inspection seems to make the same/similar mistake each time it is wrong. The problem chosen can be seen in Figure 1. Specifically, the mistake that is always made is losing a negative sign somewhere in the $-(7i)(7i)$ term and having the wrong polarity. As discussed later in the Metrics section, this gives us a way to measure draft quality irrespective of sketch and proof.

Show that for any complex number x, $x^2 + 49 = (x + 7i)(x - 7i)$.

Figure 1: Informal problem used for all evaluations

Note that all LLM calls in our experiments are calls to OpenAI's `gpt-3.5-turbo`. We use three shots for the draft, sketch, and all feedback methods. Our three methods for providing feedback to drafts were as follows.

A. Method 1: Error Feedback

Recent work on math problem-solving has shown that simply placing in the prompt a previous, incorrect response that the model has made can improve the model's solving ability Zheng et al. [2023]. We wished to test this on DSP, but there are some key differences between GSM8K-type questions and the proof generation of DSP. The primary difference is that in our setting, the success of the draft is only gauged in a downstream process; that is, a draft is first converted into a sketch, and then the sketch is passed to a formal theorem prover. The prover can only provide limited information: it can tell us whether the proof is correct or incorrect, and if it is incorrect, it can tell us the conjecture that it was trying to prove when it timed out.

We first propose passing this failed conjecture into the next draft's prompt. Such an addendum can be seen in Figure 2. We hypothesize that at the very least, this addition to the prompt will be able to inject more variance into the model's drafts, which in turn will provide for a faster proof.

Hint: your previous attempt failed when made into an Isabelle proof on the line: `have "<dots> = (x * x + (-49))"`.

Figure 2: Error Message Addendum

B. Method 2: Draft Summarization and Error Feedback

One potential shortfall of Method 1 is that even given the failed conjecture, the model will not know how this conjecture fits into its previous draft, and therefore will not know how to write a better draft. This is due to two problems: (1) in Method 1, the prompt has no access to its previous draft, rather only the failed conjecture, and (2) the failed conjecture may have failed for several reasons. A failed conjecture may arise because the model is asked to prove something that is untrue, and therefore unnecessary for the rest of the proof, or because the conjecture is too difficult, and must be broken up into smaller steps for the prover to prove it.

The ideal solution would be to insert into the prompt each previous failed conjecture *and* its respective draft. However, this is infeasible since the LLM's prompt context will not allow for more than a few failed drafts. To address this, in this method we propose using a summarizer LLM to summarize previous draft attempts, and append each summarized draft/failed conjecture pair to the next draft's prompt. We hypothesize that this will provide more information to the model that it can use to avoid repeating mistakes and address issues that have already arisen. An example addendum can be seen in Figure 3.

Hint: your previous attempt was this: "First, foil the expression to obtain $x * x + 7xi - 7xi - 49i^2$. Then simplify using the fact that $i^2 = -1$ to obtain $x * x - 49$.", but this failed in the Isabelle proof on line: `have "<dots> = (x * x + (-49))"`.

Figure 3: Draft Summary + Error Message Addendum

C. Method 3: Few-Shot Coaching

While information about failed proof attempts is easy to ascertain, it may not be useful enough to the drafter to improve future drafts. To address this, we introduce the concept of COACH, a module inspired by math competition coaches who observe a student's shortcomings, point out mistakes, suggest new techniques, and ask that they elaborate when necessary. Since the widespread use of Reinforcement Learning with Human Feedback (RLHF) to train LLM's, there have been adaptations that remove the necessity of a human. For example, the Reinforcement Learning for Feedback (RL4F) paradigm trains a "critic" model using the downstream success of the "task" model Akyurek et al. [2023]. We would like to do something similar, but because the use of RL is out of scope for this project, we propose using a few-shotted LLM as a stand-in for an RL-trained COACH.

The task for COACH is to look at a failed conjecture along with an associated sketch and draft and provide helpful feedback to the next draft attempt. COACH may also look at several summarized draft/sketch/proof attempts and provide feedback to these. To create the COACH module, as stated before we will not use RL, but will instead few-shot an LLM. To achieve this we will annotate a few incorrect (draft, sketch, proof) triplets by providing helpful and insightful feedback. These will then serve as our few shots, and we will rely on the LLM's successful in-context learning to provide good feedback. Such an addendum can be seen in Figure 4.

Hint: In a previous attempt, you made a mistake in your DRAFT when simplifying the term $7i * (-7i)$. Make sure to pay attention to the negative sign in the term $7i(-7i)$. The correct simplification should be $7i(-7i) = -49i^2$.

Figure 4: COACH-generated Addendum

D. Metrics

We use two metrics to evaluate the feedback augmentation inserted into prompts.

1) Metric 1: Normal Draft, Sketch, Prove: The most natural metric to use is to simply count the proportion of successful proofs generated by the whole DSP framework where the draft is generated normally (control) and using each feedback mechanism (treatment). This process is quite expensive because the 'prove' process must fully time out before it fails. Note that the results of standard DSP are not comparable to the original DSP paper because they use different LLM's and a slightly more sophisticated few-shot selection mechanism Jiang et al. [2022].

2) Metric 2: 'Manual' Evaluation: Our modifications only change the process in producing the draft. Therefore the issue with the first metric is that it is an indirect, downstream metric for a modification that occurs as a first step. For example, it is possible that an augmented prompt produces better quality drafts (i.e. more correct), but these drafts are more difficult to sketch/prove.

To address this issue, we cash in the luxury of testing on only one problem. Because we test only one problem, we can 'hard code' a way of checking whether the draft contains a mistake via simple string manipulation, and report the proportion that contain no mistakes. This metric is therefore much more accurate in assessing draft quality, and is also much cheaper to perform since it does not require any sketches or proofs.

III. Results

We show the results of the full DSP procedure in Table I and Figure 5, with the control and all three feedback mechanisms. As can be seen from the table, all three feedback mechanisms perform significantly worse than the control. The sample sizes were 143 for the control and 100 for each of the feedback types.

	Control	Error Message	Error Message + Draft Summary	COACH
Proof Success Rate (%)	27.27	14.0	10.0	13.0

Table I: Full DSP metric draft quality

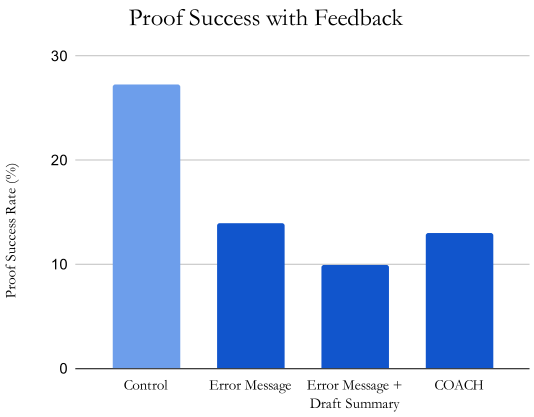


Figure 5: Full DSP metric draft quality

In Table II and Figure 6, we see the results when we evaluate the draft quality only. Here we see no difference between the control and error message feedback. Noticeably the summarized draft + error message feedback causes the draft to make far more mistakes (about 18% more mistakes). Finally, we see that the COACH feedback very slightly outperforms the control. The difference here is about 0.8%. With a sample size of 5400 for control and 2000 for COACH, using a Z-test, we can be 81.3% confident that the COACH does in fact reduce the number of mistakes in new drafts.

	Control	Error Message	Error Message + Draft Summary	COACH
Correct Draft Rate (%)	86.20	86.20	68.20	87.0

Table II: Direct metric draft quality

One important point to notice is the difference in success between the entire DSP process and the correct drafts. That is, even with a correct draft, only 14-32% of DSP runs finished with a correct proof. In other words, sketching and proving cannot be relied on to work even with a correct draft.

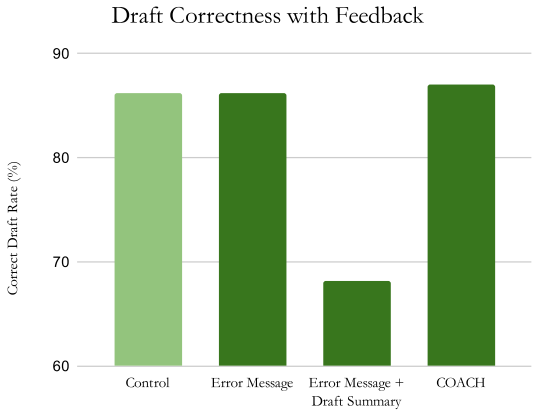


Figure 6: Direct metric draft quality

IV. Discussion

Clearly the results are not convincing in showing that any of these methods are effective at improving drafts. All three feedback mechanisms performed significantly worse in the overall DSP setting and the COACH had only a marginal effect in direct draft quality.

However, having both a direct and indirect metric allows us to make conclusions we could not make otherwise. Specifically, we see that the COACH hinders the overall DSP success rate, but is able to improve the drafts. This suggests that in some way, the COACH may improve a draft's correctness, but in doing so may also make the sketches worse. We hypothesize that this has to do with small true statements (in this case such as $i^2 = -1$), which COACH may induce the drafter to repeat which then make the causal proof sequence more difficult for the sketcher to follow.

Despite the fact that the COACH is probably very slightly improving draft quality, the difference is so small that it would not have an effect if implemented. Creating the COACH feedback requires several LLM calls and it would likely be more efficient overall to simply make more drafts than to use COACH as is.

We also note the marked decline in success when the drafter is provided with a draft summary and error message. Here we hypothesize that this is because the model is using the inccorect draft summary that it is provided as a signal for its next draft. That is, instead of using it and the error message to 'understand' its mistake, it is using it as a guide to create a new proof, and in doing so is repeating the same mistakes that are present in that draft.

All of these conclusions apply to this specific problem, and it is unclear whether it would generalize to other problems where other types of feedback could be helpful. Specifically, following the work of Madaan et al. [2023], we see that math-type feedback is the least successful, likely because the difference between a correct and incorrect math statement could be as subtle as a single character. In our problem, the COACH was correcting a minus sign to a plus sign, which is just a mistake in algebraic manipulation. Algebraic manipulation is such a simple symbolic task that it is unclear whether the positive results of feedback in works such as Akyurek et al. [2023] apply.

V. Future Work

With this key observation that incorrect algebraic manipulation may not respond well to feedback, we propose using feedback in more abstract components of proofs. That is, instead of trying to correct a minus sign to a plus sign, it is possible that the positive effects of feedback that have been studied could come into effect in concepts like proof technique, picking a correct formula to apply, interpreting a solution, etc.

In order to investigate whether feedback could aid in these concepts, it would be necessary to first solve the issue of algebraic and other symbolic manipulation. Works such as Mialon et al. [2023], Wang et al. [2023], Zhou et al. [2023], and Schick et al. [2023] provide ways in which it would be possible to integrate an algebraic solver into this DSP framework. Once this is done, it would be possible to have COACH generate more abstract feedback.

Another avenue of potential work is to break the Draft step into several layers of abstraction. For example, one could have a Brainstorm, Recall, Outline, Draft, Sketch, Prove framework, wherein each subsequent layer reduces in abstraction. Whether this hierarchy of abstraction is pertinent seems to be an open question. However, to conduct experiments that test this would likely also require a system which is capable of using tools.

References

Afra Feyza Akyurek, Ekin Akyurek, Ashwin Kalyan, Peter Clark, Derry Tanti Wijaya, and Niket Tandon. RL4F: Generating natural language feedback with reinforcement learning for repairing model outputs. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7716–7733, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.427. URL https://aclanthology.org/2023.acl-long.427.

Albert Q Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. *arXiv preprint arXiv:2210.12283*, 2022.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.

Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. Augmented language models: a survey, 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.

Ke Wang, Houxing Ren, Aojun Zhou, Zimu Lu, Sichun Luo, Weikang Shi, Renrui Zhang, Linqi Song, Mingjie Zhan, and Hongsheng Li. Mathcoder: Seamless code integration in llms for enhanced mathematical reasoning, 2023.

Chuanyang Zheng, Zhengying Liu, Enze Xie, Zhenguo Li, and Yu Li. Progressive-hint prompting improves reasoning in large language models. *arXiv preprint arXiv:2304.09797*, 2023.

Aojun Zhou, Ke Wang, Zimu Lu, Weikang Shi, Sichun Luo, Zipeng Qin, Shaoqing Lu, Anya Jia, Linqi Song, Mingjie Zhan, and Hongsheng Li. Solving challenging math word problems using gpt-4 code interpreter with code-based self-verification, 2023.